

Relaciones entre clases en Python

Del diagrama UML al código, ejemplo por ejemplo

Clase única · Asociación · Agregación · Composición

Clase abstracta · Herencia · Dependencia (uso y creación) · Interfaces

Cada relación con su multiplicidad $1 \leftrightarrow 1$ y $1 \leftrightarrow \text{muchos}$ · diagramas UML

AUTOR

Mag. en Ing. de Software · Alberto Cortez

Análisis y Diseño Orientado a Objetos · Programación

Índice

Cómo leer este manual

La anatomía de las secciones, el orden del catálogo y la notación UML

1 • La clase única

El átomo: sin relaciones, solo estado, comportamiento e invariantes

2 • Asociación

Dos clases que se conocen · 1↔1 (Conductor–Vehículo) · 1↔muchos (Profesor–Materia)

3 • Agregación

'Tiene un', sin poseer · 1↔1 (Oficina–Impresora) · 1↔muchos (Equipo–Jugador)

4 • Composición

'Se compone de' · 1↔1 (Factura–Encabezado) · 1↔muchos (Pedido–Ítem) · tabla comparativa

5 • Clase abstracta

ABC y @abstractmethod: el molde que no se puede instanciar (MedioDePago)

6 • Herencia

'Es un', sobrescritura y polimorfismo (Empleado–Gerente–Vendedor)

7 • Dependencia

El vínculo transitorio · de uso (Cajero–Ticket) · de creación (Fábrica–Reporte)

8 • Interfaces

El contrato sin implementación · ABC pura (nominal) · Protocol (estructural)

Cierre • El mapa completo

Las diez entradas del catálogo en una sola tabla de referencia

Cómo leer este manual

El diagrama manda; el código obedece

Este manual es complementario de *De Java a Python · POO por equivalencia*. Aquel libro defendía una tesis: el modelo UML no cambia cuando cambia el lenguaje. Este la da por probada y baja a la práctica. Su objeto es más acotado y más operativo: recorrer, una por una, **todas las relaciones que un diagrama de clases puede expresar, y mostrar cómo cada una se materializa en Python**. No es un tratado de UML ni un curso de Python; es un puente entre los dos, transitado en un solo sentido —del dibujo al código— porque ese es el sentido en el que un estudiante o un profesional realmente trabaja: primero entiende el dominio y lo modela, y recién después escribe.

Cada sección tiene la misma anatomía, y esa repetición es deliberada. Abre con un **párrafo narrativo** que explica qué relación del mundo real se está modelando y por qué. Sigue con el **diagrama UML** que la captura. Continúa con el **código Python mínimo** que la implementa, sin una línea de más. Y cierra con una caja que responde la pregunta que el código no responde por sí solo: *¿por qué esta relación y no otra?* El lector que recorra las ocho secciones en orden habrá visto el catálogo completo; el que venga a buscar una relación puntual encontrará todo lo que necesita en dos páginas.

El orden no es alfabético ni arbitrario: sigue la **fuerza del vínculo**, de menor a mayor. Primero la clase que no se relaciona con nadie —el átomo—. Después las tres relaciones estructurales, ordenadas por cuánto compromete una clase a la otra: la **asociación** (dos clases que apenas se conocen), la **agregación** (una agrupa a la otra, pero sin poseerla), y la **composición** (una contiene a la otra y la posee tan completamente que comparten destino). Luego las relaciones de **herencia** —la clase abstracta y la generalización—, que expresan parentesco conceptual. Y al final las más sutiles y transitorias: las **dependencias** de uso y de creación, y las **interfaces**, que son contratos antes que vínculos.

LA DISTINCIÓN QUE MÁS SE CONFUNDE

Hay una trampa que atraviesa todo el manual y conviene desactivarla desde el principio. Las tres relaciones estructurales —asociación, agregación y composición— se escriben en Python de forma **casi idéntica**: una referencia guardada en un atributo. Si uno solo mira el tipo de dato, las tres parecen la misma cosa. La diferencia no vive en la sintaxis: vive en **el ciclo de vida**, es decir, en quién crea al objeto contenido y quién lo destruye.

De ahí sale una regla práctica que vale la pena memorizar. Si el todo *recibe* la parte ya construida desde afuera —como parámetro—, es asociación o agregación: la parte tenía vida propia antes de entrar. Si el todo *fabrica* la parte adentro, en su propio constructor, y esa parte no tendría sentido existiendo sola, es composición. El diagrama codifica esta decisión con un símbolo —el rombo vacío ◇ para la agregación, el rombo lleno ◆ para la composición—; el código la codifica con **el lugar donde aparece el constructor de la parte**. Aprender a leer los constructores es aprender a leer las relaciones.

Notación que se usa en los diagramas

Los diagramas de este manual están hechos en notación UML estándar. La siguiente tabla resume los símbolos que aparecen, para que ninguno quede sin traducir. Vale la pena tenerla presente en las primeras secciones; después se vuelven familiares y la tabla sobra.

Símbolo en el diagrama	Qué significa
Línea simple —	Asociación: las clases se conocen
Rombo vacío ◇—	Agregación: 'tiene un' — la parte sobrevive al todo

Símbolo en el diagrama	Qué significa
Rombo lleno 	Composición: 'se compone de' — la parte muere con el todo
Triángulo hueco 	Generalización / herencia: 'es un'
Triángulo punteado 	Realización: la clase implementa una interfaz
Flecha punteada 	Dependencia: 'usa' o 'crea' de forma transitoria
1 0..1 0..*	Multiplicidad: cuántos objetos de cada lado
« abstract » « interface »	Estereotipo: naturaleza especial de la clase

1 · La clase única

El átomo: sin relaciones, solo estado y comportamiento

Antes de que existan relaciones, existe la clase. Una clase que no se vincula con ninguna otra es el punto de partida de todo diseño orientado a objetos, y modela lo más simple que el dominio puede ofrecer: un concepto **autónomo**, que se explica enteramente por sí mismo. Un termostato es un buen ejemplo. Tiene un **estado** —la temperatura actual, y los límites de seguridad entre los que puede moverse— y un **comportamiento** —puede subir, bajar, y responder si está dentro de rango—. Nada de eso requiere conocer otra clase.

Lo interesante de una clase aislada no es lo que se relaciona con afuera, sino cómo se protege por dentro. El termostato tiene una regla que siempre debe cumplirse: la temperatura nunca puede quedar fuera de los límites. Esa regla es un **invariante**, y el mejor lugar para hacerla cumplir es el constructor, porque el constructor es el único momento en que un objeto pasa de no existir a existir. Si no se lo puede construir en un estado válido, sencillamente no se lo construye. Todo lo demás —los métodos *subir* y *bajar*— se limita después a *mantener* ese invariante que el constructor *estableció*.

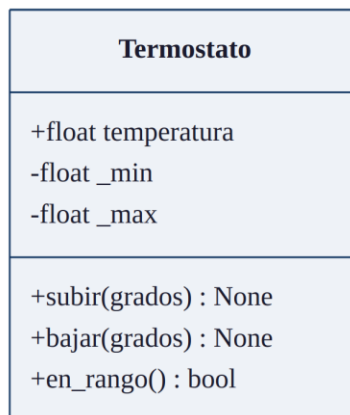


Figura 1 · La clase Termostato. Sin relaciones: solo atributos, métodos y el invariante que protege.

Python

```
class Termostato:
    """Un termostato con límites de seguridad. No se relaciona con nadie."""

    def __init__(self, temperatura: float = 20.0,
                  minimo: float = 10.0, maximo: float = 30.0) -> None:
        if not minimo <= temperatura <= maximo:
            raise ValueError("temperatura inicial fuera de rango")
        self._min = minimo
```

```

self._max = maximo
self.temperatura = temperatura      # atributo público

def subir(self, grados: float) -> None:
    self.temperatura = min(self.temperatura + grados, self._max)

def bajar(self, grados: float) -> None:
    self.temperatura = max(self.temperatura - grados, self._min)

@property
def en_rango(self) -> bool:          # atributo derivado (/en_rango)
    return self._min <= self.temperatura <= self._max

def __repr__(self) -> str:
    return f"Termostato({self.temperatura}°C)"

```

Lo que hay que notar

El invariante `minimo <= temperatura <= maximo` se valida **en el constructor**. Esta es la traducción directa de la restricción *{ constraint }* que en el diagrama UML no se dibuja como una caja sino como una nota: una regla del dominio que el código debe garantizar pero que no tiene forma gráfica propia. Ponerla en el constructor asegura que ningún termostato pueda nacer roto.

`en_rango` es una **property**, no un método común. En el diagrama sería un atributo derivado —el que se marca con una barra, `/en_rango`—: algo que se *lee* como un atributo pero se *calcula* en el momento a partir de otros. Es el primer ejemplo de un patrón que reaparecerá en casi todas las secciones siguientes.

2 • Asociación

Dos clases que se conocen — el vínculo más débil

La asociación es la relación más suave que dos clases pueden tener sin ser indiferentes. Expresa que dos objetos **independientes** necesitan conocerse para colaborar, pero que **ninguno es dueño del otro**. Cada uno existe por su cuenta, se crea por su lado, y puede seguir viviendo aunque el otro desaparezca. Es una relación entre iguales: no hay un todo y una parte, hay dos entidades que se dan la mano.

En el código, esta relación entre iguales se traduce en algo muy concreto: cada clase guarda una **referencia** a la otra en un atributo, y esas referencias se establecen *después* de que ambos objetos ya existen. El detalle revelador —el que distingue a la asociación de las relaciones más fuertes que veremos luego— es que **ningún objeto aparece dentro del constructor del otro**. Se crean por separado y se presentan en un segundo paso.

2.1 • Asociación 1 ↔ 1

El caso más simple es el de dos objetos que se corresponden uno a uno. Un **conductor** tiene asignado un **vehículo**, y ese vehículo tiene asignado un conductor. Pensemos en el mundo real: el vehículo estaba en el concesionario, con patente y todo, mucho antes de que este conductor apareciera; y si el conductor se va, el vehículo no se desintegra, simplemente queda libre para otro. La relación los vincula, pero no los fusiona.

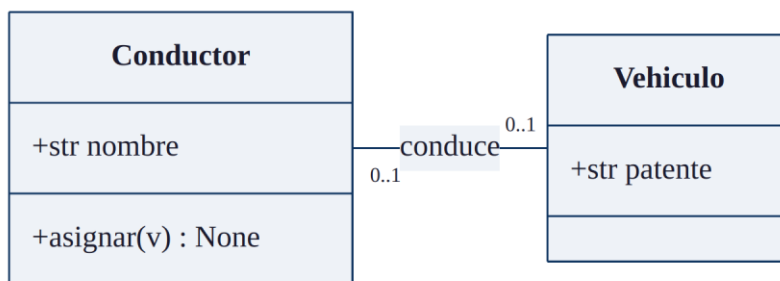


Figura 2 · Asociación 1 a 1. Línea simple, multiplicidad 0..1 en ambos extremos: cada uno puede tener, a lo sumo, un par asignado.

Python — asociación 1 a 1

```

class Vehiculo:
    def __init__(self, patente: str) -> None:
        self.patente = patente
        self.conductor: 'Conductor | None' = None # lado inverso, opcional

class Conductor:
    def __init__(self, nombre: str) -> None:
        self.nombre = nombre
        self.vehiculo: 'Vehiculo | None' = None

    def asignar(self, v: Vehiculo) -> None:
        self.vehiculo = v
        v.conductor = self # mantiene la relación bidireccional

# Uso: los dos se crean por separado y DESPUÉS se vinculan
juan = Conductor("Juan")
auto = Vehiculo("ABC123")
juan.asignar(auto) # ahora se conocen
  
```

La marca de la asociación

Antes de mirar los constructores, mirá los type hints: `vehiculo: 'Vehiculo | None'` y `conductor: 'Conductor | None'`. Ese `| None` es la **traducción literal de la multiplicidad 0..1** del diagrama: recién creado, un conductor no tiene vehículo, y un vehículo no tiene conductor. La opcionalidad que el diagrama dibuja como '0..1', el código la escribe como 'o el objeto, o None'.

Mirá las últimas tres líneas del ejemplo. El conductor se construye con `Conductor(...)` y el vehículo con `Vehiculo(...)`, en líneas **separadas e independientes**. Recién en una tercera línea se los vincula con `asignar`. Ese 'se crean afuera y después se presentan' es exactamente lo que hace que esto sea una asociación y no una composición. Si en cambio el conductor fabricara su propio vehículo en el constructor, estaríamos ante una relación mucho más fuerte.

El método `asignar` actualiza **los dos lados** de la relación: setea `self.vehiculo` y también `v.conductor`. Esto es lo que mantiene la asociación coherente en ambos sentidos —un error clásico es actualizar solo un lado y que el otro quede desincronizado—.

2.2 · Asociación 1 ↔ muchos

La misma relación se vuelve más rica cuando de un lado hay muchos. Un **profesor** dicta varias **materias**; cada materia tiene un profesor a cargo. Las materias existen en el plan de estudios con independencia de quién las dé: si el profesor renuncia, POO y Redes siguen figurando en el plan, esperando un nuevo docente. Del lado del profesor hay *una colección* de materias; del lado de la

materia, *una sola* referencia al profesor. Esa asimetría —uno contra muchos— es la multiplicidad, y en el código se refleja con una lista de un lado y una referencia simple del otro.

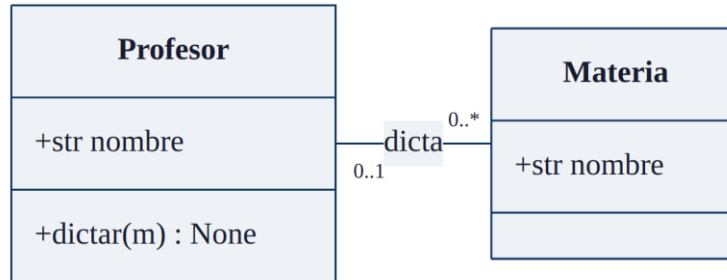


Figura 3 · Asociación 1 a muchos. Un profesor dicta cero o más materias; cada materia tiene, a lo sumo, un profesor (0..1).

Python — asociación 1 a muchos

```
class Materia:
    def __init__(self, nombre: str) -> None:
        self.nombre = nombre
        self.profesor: 'Profesor | None' = None

class Profesor:
    def __init__(self, nombre: str) -> None:
        self.nombre = nombre
        self._materias: list[Materia] = []      # lado 'muchos'

    def dictar(self, m: Materia) -> None:
        self._materias.append(m)
        m.profesor = self

    @property
    def materias(self) -> list[Materia]:
        return list(self._materias)            # copia defensiva

# Uso
p = Profesor("Cortez")
poo = Materia("POO"); redes = Materia("Redes")
p.dictar(poo); p.dictar(redes)
# Si el profesor se va, poo y redes SIGUEN existiendo (solo pierden ref)
```

Detalle de implementación

La property `materias` no devuelve la lista interna: devuelve `list(self._materias)`, que es una **copia**. El motivo es de disciplina: si devolviéramos la lista real, cualquiera desde afuera podría hacer `profesor.materias.append(x)` y agregar una materia salteándose el método `dictar`, que es el único lugar donde la relación se mantiene consistente en los dos sentidos. La copia defensiva cierra esa puerta trasera.

3 · Agregación

'Tiene un' — el todo agrupa a la parte, pero no la posee

La agregación es una asociación con un matiz de pertenencia. Ya no son dos iguales que se conocen: ahora hay un **todo** que *agrupa* a sus **partes**. Pero es una pertenencia débil, y el rombo vacío del diagrama insiste en esa debilidad: las partes **existían antes de entrar al todo y le sobreviven cuando salen**. El todo las junta, las coordina, pero no las engendra ni las mata.

La imagen que mejor lo captura es la de un equipo y sus jugadores. Un jugador es una persona con carrera propia: fichó, jugó, se transfirió, seguirá jugando en otro club. El equipo lo *agrupa* durante un tiempo, pero disolver el equipo no borra a los jugadores del mundo —simplemente quedan libres—. En el código, esto se manifiesta en un lugar preciso: la parte **se recibe ya construida**, como parámetro, y no se fabrica adentro. La firma del constructor delata la relación.

3.1 · Agregación 1 ↔ 1

Una **oficina** tiene asignada una **impresora**. La impresora fue comprada por la empresa, tiene su número de serie, y mañana puede reasignarse a otra oficina sin dejar de ser la misma impresora. No nació con la oficina ni muere con ella: apenas está *asignada* a ella. Por eso, en el código, la oficina la **recibe** en su constructor en lugar de crearla.

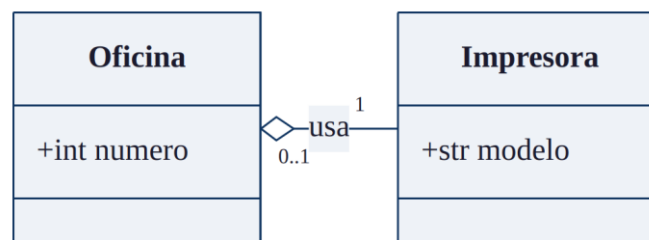


Figura 4 · Agregación 1 a 1. El rombo vacío del lado de la oficina: cada oficina usa exactamente una impresora; la impresora puede estar sin asignar (0..1).

Python — agregación 1 a 1

```

class Impresora:
    def __init__(self, modelo: str) -> None:
        self.modelo = modelo

class Oficina:
    def __init__(self, numero: int, impresora: Impresora) -> None:
        self.numero = numero
        self.impresora = impresora      # RECIBE la impresora ya creada

# Uso: la impresora se crea AFUERA y se pasa al constructor
hp = Impresora("HP LaserJet")
of1 = Oficina(101, hp)
del of1                                # la oficina desaparece...
print(hp.modelo)                       # ...la impresora sigue viva
of2 = Oficina(102, hp)                 # ...y se reasigna a otra oficina

```

La firma del constructor delata la agregación

El dato decisivo está en la firma: `def __init__(self, numero, impresora: Impresora)`. La impresora entra como **parámetro**. El todo no la fabrica, la **recibe**. Por eso, cuando ejecutamos `del of1` y la oficina desaparece, la impresora `hp` sigue perfectamente viva, y queda disponible para reasignarse a otra oficina. Compará esta firma con la de la composición en la sección 4: ahí la parte no aparece en la firma, porque se crea adentro.

3.2 · Agregación 1 ↔ muchos

Subamos a la multiplicidad de muchos. Un **equipo** agrupa a varios **jugadores**, y este es el caso canónico de la agregación, el que conviene tener en la cabeza como imagen de referencia. Los jugadores entran y salen por transferencias; el equipo es apenas el marco que los reúne en un momento dado. La prueba definitiva de que es agregación y no composición es la operación de **transferir**: un jugador

puede pasar de un equipo a otro *sin ser destruido en el camino*. Migra. Y la capacidad de la parte de migrar de un todo a otro es, en el fondo, la definición operativa de la agregación.

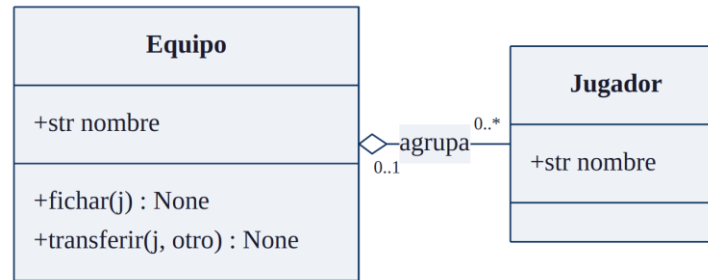


Figura 5 · Agregación 1 a muchos. Un equipo agrupa cero o más jugadores; cada jugador está, a lo sumo, en un equipo y puede migrar.

Python — agregación 1 a muchos

```
class Jugador:
    def __init__(self, nombre: str) -> None:
        self.nombre = nombre

class Equipo:
    def __init__(self, nombre: str) -> None:
        self.nombre = nombre
        self._jugadores: list[Jugador] = []

    def fichar(self, j: Jugador) -> None:    # recibe jugadores externos
        self._jugadores.append(j)

    def transferir(self, j: Jugador, otro: 'Equipo') -> None:
        self._jugadores.remove(j)
        otro.fichar(j)                    # el jugador NO se destruye

# Uso
messi = Jugador("Messi")
a = Equipo("A"); b = Equipo("B")
a.fichar(messi)
a.transferir(messi, b)                  # cambia de equipo, sigue existiendo
```

Prueba del rombo vacío

El método `transferir` es la evidencia en código de que estamos ante una agregación. Saca al jugador de la lista de un equipo (`remove`) y lo mete en la de otro (`fichar`), **sin destruirlo en ningún momento**. El mismo objeto `messi` existe antes, durante y después de la transferencia. Un jugador puede recorrer tres o cuatro clubes en su carrera siendo siempre el mismo. Que la parte pueda saltar de un todo a otro es lo que el rombo vacío del diagrama está afirmando gráficamente.

4 · Composición

'Se compone de' — la parte nace y muere con el todo

La composición es el vínculo estructural más fuerte que existe, y el rombo lleno del diagrama lo anuncia con firmeza. Aquí la parte ya no es independiente: **no tiene sentido fuera del todo y su ciclo de vida está atado por completo al del todo**. El todo no recibe a la parte: la **fabrica adentro**, en su propio constructor. Y cuando el todo desaparece, la parte desaparece con él, sin dejar rastro, porque nunca tuvo existencia autónoma que reclamar.

La imagen clásica es la de una casa y sus habitaciones. Una habitación no es una entidad que ande por el mundo esperando ser asignada a una casa; nace *como parte de* una casa concreta, y si la casa se demuele, no quedan habitaciones flotando en el aire buscando otro edificio. Esa dependencia total de existencia es lo que separa la composición de la agregación. Y en el código la diferencia se reduce a una sola cosa, que conviene subrayar hasta el cansancio porque es *la* distinción que más se confunde: en la agregación la parte **entra por el constructor como parámetro**; en la composición la parte **se crea dentro del constructor**. Mismo atributo, decisión opuesta.

4.1 · Composición 1 ↔ 1

Una **factura** se compone de un **encabezado**: los datos de cabecera —número, cliente, fecha— que toda factura lleva arriba. Ese encabezado no existe suelto en ninguna parte; no tiene sentido hablar de 'un encabezado' sin la factura a la que pertenece. No se comparte con otras facturas ni sobrevive a la suya. Por eso la factura lo **construye en su propio constructor**, a partir de los datos que recibe, en lugar de aceptar un encabezado ya hecho desde afuera.

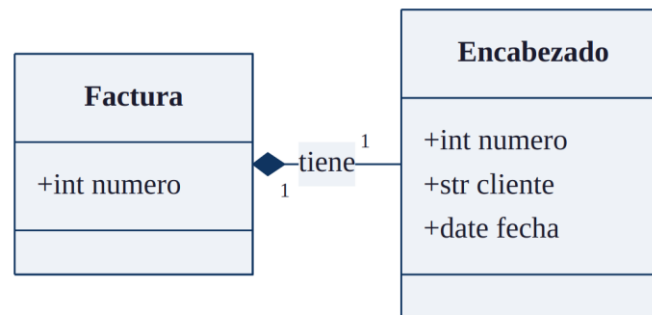


Figura 6 · Composición 1 a 1. El rombo lleno: el encabezado nace con la factura y muere con ella.

Python — composición 1 a 1

```

from datetime import date

class Encabezado:
    def __init__(self, numero: int, cliente: str) -> None:
        self.numero = numero
        self.cliente = cliente
        self.fecha = date.today()

class Factura:
    def __init__(self, numero: int, cliente: str) -> None:
        # La factura CREA su encabezado adentro. No lo recibe.
        self._encabezado = Encabezado(numero, cliente)

    @property
    def encabezado(self) -> Encabezado:
        return self._encabezado

# Uso: no se crea un Encabezado 'suelto' y se pasa; nace dentro de la Factura
f = Factura(1001, 'ACME S.A.')      # el encabezado nace acá adentro
# cuando 'f' desaparece, su encabezado desaparece con ella
  
```

La diferencia con la agregación, en una línea

Agregación (3.1): `def __init__(self, impresora):` → la parte **entra por parámetro** (se recibe).

Composición (4.1): `self._encabezado = Encabezado(...)` → la parte **se crea adentro** (se fabrica).

Fijate que el *almacenamiento* es idéntico —en ambos casos un atributo guarda una referencia—. Lo único que cambia es el *origen* del objeto guardado. Esa diferencia mínima en el código refleja una diferencia enorme en el modelo: en un caso las vidas son independientes, en el otro están fundidas. **El constructor es la prueba.**

4.2 · Composición 1 ↔ muchos

Un **pedido** se compone de muchos **ítems**. Un ítem de pedido —dos unidades de tal producto, a tal precio— no es un objeto que exista por fuera del pedido que lo contiene; es una línea de ese pedido y nada más. Borrado el pedido, sus ítems no tienen adónde ir. El pedido los **fabrica** a partir de datos crudos, mediante un método `agregar` que recibe producto, cantidad y precio, y construye el ítem internamente. Nunca recibe un ítem ya hecho, porque un ítem ya hecho, huérfano de pedido, sería un sinsentido.

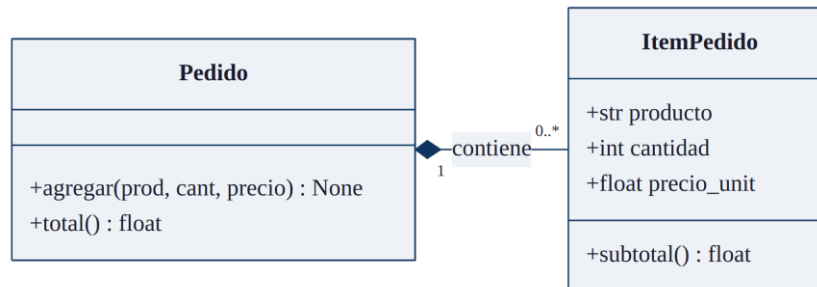


Figura 7 · Composición 1 a muchos. El pedido contiene cero o más ítems, que fabrica internamente y que mueren con él.

Python — composición 1 a muchos

```
from dataclasses import dataclass

@dataclass(frozen=True)          # value object: sin identidad propia
class ItemPedido:
    producto: str
    cantidad: int
    precio_unit: float

    @property
    def subtotal(self) -> float:
        return self.cantidad * self.precio_unit

class Pedido:
    def __init__(self) -> None:
        self._items: list[ItemPedido] = []

    def agregar(self, producto: str, cantidad: int, precio: float) -> None:
        # El pedido CREA el ítem a partir de datos crudos. No lo recibe hecho.
        self._items.append(ItemPedido(producto, cantidad, precio))

    @property
    def total(self) -> float:
        return sum(i.subtotal for i in self._items)  # comprehension

# Uso
ped = Pedido()
ped.agregar('Teclado', 2, 15000.0)  # el ítem nace acá adentro
ped.agregar('Mouse', 1, 8000.0)
```

```
print(ped.total)
```

```
# 38000.0
```

Por qué acá aparece `@dataclass(frozen=True)`

El `ItemPedido` es un **objeto-valor**: se define enteramente por su contenido —el mismo producto, la misma cantidad y el mismo precio *son* el mismo ítem—, no tiene identidad propia ni ciclo de vida más allá de su pedido, y no debería mutar una vez creado. Todo eso lo vuelve el candidato perfecto para `@dataclass(frozen=True)`, que le da comparación por contenido y lo hace inmutable de verdad. Es la misma categoría que `Lado` o `Medida` del manual anterior.

El `Pedido`, en cambio, es una **entidad**: tiene comportamiento, cambia a lo largo del tiempo (se le agregan ítems), y su identidad no depende de su contenido —dos pedidos con los mismos ítems siguen siendo dos pedidos distintos—. Por eso `Pedido` no va como `dataclass`. Esta distinción entre objeto-valor y entidad no la inventó Python: es la misma que aplicabas en el modelado, en cualquier lenguaje.

Resumen • asociación vs. agregación vs. composición

Antes de pasar a las relaciones de herencia, conviene detenerse. Las tres relaciones estructurales que acabamos de recorrer son **las que más se confunden entre sí**, precisamente porque en el código se parecen tanto. La siguiente tabla las pone lado a lado en los cuatro criterios que las distinguen. Si alguna vez dudás cuál estás modelando, la fila que resuelve la duda es casi siempre la primera: *¿quién crea la parte?*.

Criterio	Asociación	Agregación ◇	Composición ◆
¿Quién crea la parte?	Cualquiera, afuera	Afuera, se recibe	El todo, adentro
¿La parte sobrevive al todo?	Sí	Sí, migra a otro	No, muere con el todo
Cómo entra al código	Referencia mutua	Parámetro del ctor	Parte(...) en el ctor
Ejemplo de este manual	Profesor—Materia	Equipo—Jugador	Pedido—Ítem

5 • Clase abstracta

Un molde que no se puede instanciar hasta completarlo

Dejamos las relaciones estructurales y entramos en las de herencia, que expresan parentesco conceptual en lugar de contención física. La primera de ellas es la clase abstracta, que en rigor no es una relación sino un tipo especial de clase: una que define **qué operaciones deben existir** sin comprometerse a resolverlas todas. Es un molde incompleto, un contrato parcial que las subclasses están obligadas a terminar.

Pensemos en un medio de pago. Sabemos que todo medio de pago debe poder *pagar* un monto y decir su *nombre*, pero *cómo* se paga depende de si es efectivo, tarjeta o transferencia —y esas formas concretas no se pueden anticipar en el molde general—. La clase `MedioDePago` declara entonces esas dos operaciones como **abstractas** (con el decorador `@abstractmethod`) y hereda de `ABC`. La consecuencia es contundente: una clase con al menos un método abstracto sin implementar **no se puede instanciar**. Python rechaza construirla. Y ese rechazo, lejos de ser una molestia, es el mayor valor de la clase abstracta.

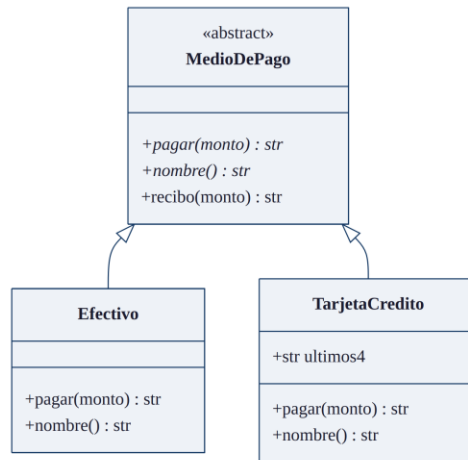


Figura 8 · Clase abstracta. MedioDePago (estereotipo «abstract», métodos abstractos con *) y sus dos realizaciones concretas.

Python — clase abstracta + subclasses concretas

```

from abc import ABC, abstractmethod

class MedioDePago(ABC):
    @abstractmethod
    def pagar(self, monto: float) -> str: ...

    @abstractmethod
    def nombre(self) -> str: ...

    def recibo(self, monto: float) -> str:      # método concreto compartido
        return f'[{self.nombre()}] {self.pagar(monto)}'

class Efectivo(MedioDePago):
    def pagar(self, monto: float) -> str:
        return f'Pagado ${monto} en efectivo'
    def nombre(self) -> str:
        return 'Efectivo'

class TarjetaCredito(MedioDePago):
    def __init__(self, ultimos4: str) -> None:
        self.ultimos4 = ultimos4
    def pagar(self, monto: float) -> str:
        return f'Pagado ${monto} con tarjeta ****{self.ultimos4}'
    def nombre(self) -> str:
        return 'Tarjeta'

# MedioDePago() -> TypeError: no se puede instanciar (faltan métodos)
print(Efectivo().recibo(500))          # [Efectivo] Pagado $500 en efectivo
  
```

El valor de la ABC: el error llega temprano

Supongamos que alguien escribe una subclase `Transferencia` y se olvida de implementar `pagar`. Con una clase abstracta bien hecha, Python **no la deja construir**: tira un `TypeError` en el momento de *instanciar*, y el mensaje nombra exactamente el método que falta. El error aparece justo donde está la causa. Sin la ABC, ese olvido no se notaría hasta que alguien llamara al método —quizás en producción, quizás tres módulos más lejos, con un error mucho más difícil de rastrear—. La clase abstracta **acorta la distancia entre la causa y el síntoma**, y esa es una razón de diseño legítima, no un capricho.

Notá además que `recibo` es un método **concreto**, con implementación real, que vive en la clase abstracta y todas las subclases heredan. Una clase abstracta no es solo un conjunto de firmas vacías: puede aportar comportamiento común. Esa capacidad de compartir código es, como veremos en la sección 8, la razón principal para preferir una ABC frente a una interfaz pura.

6 · Herencia

'Es un' — la generalización que el dominio afirma

La herencia expresa la relación más intuitiva del diseño orientado a objetos: la relación '**es-un**'. Un gerente *es un* empleado; un vendedor *es un* empleado. Ambos comparten lo que todo empleado tiene —un nombre, un sueldo base— y lo reutilizan, pero cada uno especializa ese concepto general a su manera. La herencia permite escribir una vez lo común, en la clase padre, y dejar que las subclases agreguen o modifiquen lo que las distingue.

El caso del sueldo lo ilustra bien. Todo empleado tiene un método `sueldo`, pero el gerente le suma un bono y el vendedor una comisión. Ambos **sobrescriben** el método del padre para incorporar su particularidad. Y acá aparece la magia del polimorfismo: podemos recorrer una lista mezclada de gerentes y vendedores llamando a todos `sueldo`, y cada objeto responde según lo que *realmente es*, sin que el código que recorre la lista necesite saber de qué tipo es cada uno. En Python, esta relación se escribe poniendo la superclase entre paréntesis, y tiene dos particularidades que a quien viene de Java le tienden trampas.

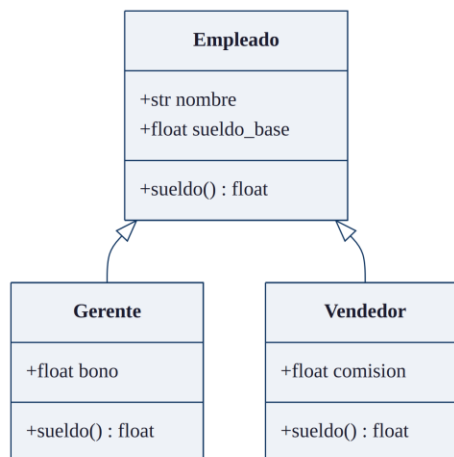


Figura 9 · Herencia. *Empleado* como generalización; *Gerente* y *Vendedor* especializan y sobrescriben `sueldo()`.

Python — herencia y sobreescritura

```
class Empleado:
    def __init__(self, nombre: str, sueldo_base: float) -> None:
        self.nombre = nombre
        self.sueldo_base = sueldo_base

    def sueldo(self) -> float:
        return self.sueldo_base

class Gerente(Empleado):
    def __init__(self, nombre: str, sueldo_base: float, bono: float) -> None:
        super().__init__(nombre, sueldo_base) # PRIMERA línea, sin excepción
        self.bono = bono
```

```

def sueldo(self) -> float:
    return self.sueldo_base + self.bono # sobrescribe

class Vendedor(Empleado):
    def __init__(self, nombre: str, base: float, comision: float) -> None:
        super().__init__(nombre, base)
        self.comision = comision

    def sueldo(self) -> float:
        return self.sueldo_base + self.comision

# Polimorfismo: mismo mensaje, respuesta según el tipo real
nomina = [Gerente('Ana', 1000, 500), Vendedor('Beto', 800, 300)]
print([e.sueldo() for e in nomina]) # [1500, 1100]

```

Las dos trampas del pasaje, juntas en este ejemplo

super().__init__() no es automático. En Java, el constructor del padre se llama solo si uno no lo hace. En Python, no: si el hijo define `__init__` y se olvida de llamar a `super().__init__(...)`, los atributos del padre —`nombre`, `suelo_base`— nunca se inicializan, y el `AttributeError` estalla mucho después, en el primer método que intente leerlos. La regla, sin excepciones: la primera línea del constructor del hijo es la llamada al del padre.

No existe @Override. Si en lugar de `suelo` escribieras `suledo` (un simple typo), no estarías sobrescribiendo nada: estarías creando un método *nuevo*. El del padre seguiría respondiendo, tu versión nunca se llamaría, y no habría ningún error que te avise. Desde Python 3.12 podés poner `@override` sobre el método para que la herramienta `mypy` verifique que efectivamente sobrescribe algo.

7 • Dependencia

'Usa' y 'crea' — el vínculo más transitorio de todos

Llegamos al vínculo más débil y fugaz del catálogo. Una dependencia existe cuando una clase necesita a otra apenas por un instante —dentro de un método— y no la guarda como parte de su estado. No hay atributo que las una: el objeto usado aparece, cumple su función, y se esfuma cuando el método termina. En el diagrama se dibuja con una flecha **punteada**, y esa línea entrecortada expresa gráficamente lo efímero del contacto. Hay dos sabores de dependencia, y se distinguen por **de dónde viene** el objeto que se usa: si llega desde afuera o si se fabrica en el momento.

7.1 • Dependencia de USO

En la dependencia de uso, la clase **recibe** el objeto ya construido —típicamente como parámetro de un método—, lo utiliza para lo que necesita, y lo suelta. No lo crea ni lo almacena. Un **cajero** que cobra un **ticket** es el ejemplo perfecto: el ticket le llega, el cajero lee su total y emite el cobro, y ahí termina la relación. El cajero no *tiene* un ticket como parte de su identidad; apenas *usó* uno durante un método.

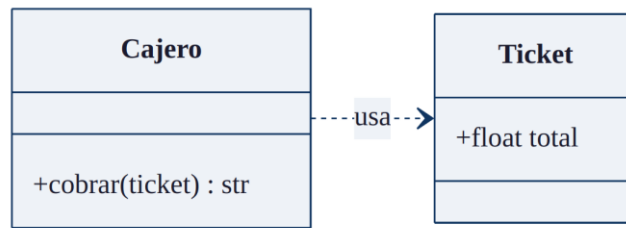


Figura 10 · Dependencia de uso. El cajero recibe el ticket, lo usa y lo suelta: no lo guarda.

Python — dependencia de uso

```

class Ticket:
    def __init__(self, total: float) -> None:
        self.total = total

class Cajero:
    def cobrar(self, ticket: Ticket) -> str: # RECIBE el ticket, lo usa
        # 'ticket' vive solo durante este método; no se guarda en self
        return f'Cobrando ${ticket.total}. Gracias.'

# Uso: el ticket se crea afuera y se PASA al método
t = Ticket(1200.0)
print(Cajero().cobrar(t))          # el Cajero no 'tiene' un ticket
  
```

Cómo se reconoce el uso

Dos señales, juntas, delatan la dependencia de uso. Primero, el objeto entra como **parámetro de un método** —no del constructor—. Segundo, y decisivo, **nunca se asigna a un self. atributo**. Si buscás la palabra `ticket` en el resto de la clase `Cajero`, no la vas a encontrar en ningún otro lado: existió únicamente durante la ejecución de `cobrar` y se desvaneció al terminar. Esa ausencia de rastro persistente es la marca del uso.

7.2 · Dependencia de CREACIÓN

La dependencia de creación es la hermana de la de uso: en lugar de recibir el objeto, la clase lo **fabrica** dentro de un método, lo usa o lo devuelve, y no lo retiene. Una **fábrica de reportes** que produce un **reporte** y lo entrega es el caso típico. La distinción fina —y la que conviene tener clarísima— es respecto de la composición: también en la composición el todo fabrica objetos, pero **los guarda como parte permanente de su estado**. En la dependencia de creación, en cambio, el objeto creado **se produce y se suelta**: la fábrica no queda 'teniendo' el reporte, solo lo generó y lo pasó a quien lo pidió.

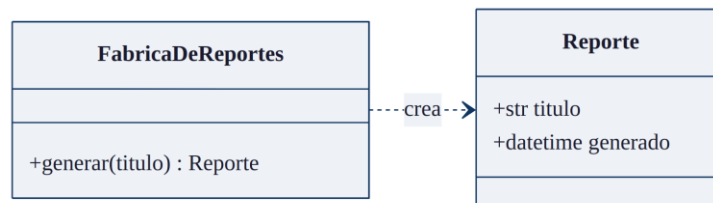


Figura 11 · Dependencia de creación. La fábrica crea el reporte dentro de un método y lo devuelve; no lo guarda.

Python — dependencia de creación

```

from datetime import datetime

class Reporte:
  
```

```

def __init__(self, titulo: str, generado: datetime) -> None:
    self.titulo = titulo
    self.generado = generado

class FabricaDeReportes:
    def generar(self, titulo: str) -> Reporte:
        # CREA el Reporte adentro y lo DEVUELVE. No lo guarda en self.
        return Reporte(titulo, datetime.now())

# Uso: la fábrica produce el reporte; no lo retiene
fab = FabricaDeReportes()
r = fab.generar('Ventas Q3')      # el reporte nace en generar() y sale
# fab no tiene ningún atributo 'reporte'

```

Uso, creación y composición — la escalera de intimidad

Uso (7.1): el objeto *llega* por parámetro de método, se usa, se descarta. La clase nunca lo tuvo.

Creación (7.2): el objeto *se fabrica* en un método, se devuelve, no se guarda. La clase lo tuvo un instante.

Composición (sección 4): el objeto *se fabrica en el constructor* y se guarda como parte permanente del estado. La clase lo tiene para siempre.

Las tres involucran fabricar o manipular otro objeto, pero forman una escalera de intimidad creciente. La pregunta que separa la creación de la composición es una sola: **¿el objeto fabricado se asigna a un self. y persiste? Si persiste, es composición. Si se devuelve y se olvida, es apenas una dependencia.**

8 • Interfaces

El contrato sin la implementación — dos formas en Python

Cerramos el catálogo con las interfaces, que más que un vínculo entre objetos son un **contrato de capacidades**: la promesa de que una clase 'sabe hacer' cierta cosa, sin decir cómo. Python no tiene la palabra reservada `interface` de Java, pero ofrece **dos maneras distintas** de expresar ese contrato, y elegir entre ellas es una de las decisiones de diseño más finas del pasaje. La diferencia entre las dos se reduce a una pregunta: *¿la clase que cumple el contrato tiene que saber que el contrato existe?*.

8.1 • Interfaz con ABC pura (contrato nominal)

La primera forma es una **ABC** con *solamente* métodos abstractos, sin nada de implementación. Es el equivalente más literal de una interfaz Java, y su rasgo definitorio es que la clase que la cumple **debe declarar explícitamente la herencia**: escribe `class EmailNotif(Notificador)`. Por eso se dice que es un contrato **nominal** —el que cumple tiene que *nombrar* lo que cumple—. Un notificador que envía por email y otro que envía por SMS declaran, cada uno, que *son* notificadores, y esa declaración los obliga a proveer el método `enviar`.

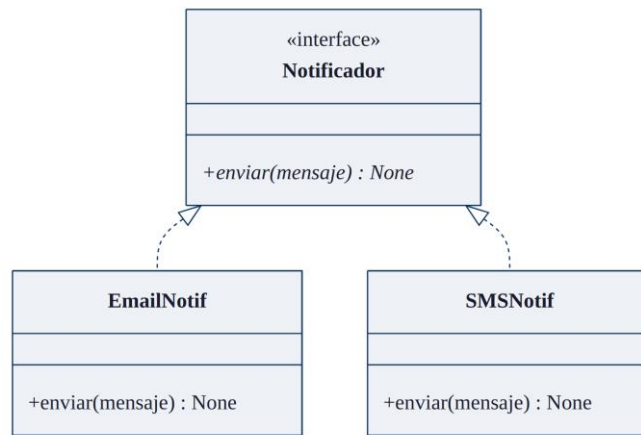


Figura 12 · Interfaz vía ABC pura. Realización (triángulo punteado): cada clase declara que implementa Notificador.

Python — interfaz vía ABC pura

```

from abc import ABC, abstractmethod

class Notificador(ABC):          # interfaz: solo firmas
    @abstractmethod
    def enviar(self, mensaje: str) -> None: ...

class EmailNotif(Notificador):   # DECLARA que la cumple
    def enviar(self, mensaje: str) -> None:
        print(f'[email] {mensaje}')

class SMSNotif(Notificador):
    def enviar(self, mensaje: str) -> None:
        print(f'[sms] {mensaje}')

def avisar(n: Notificador, msg: str) -> None:
    n.enviar(msg)                # cualquier Notificador sirve

avisar(EmailNotif(), 'Hola')
avisar(SMSNotif(), 'Hola')
  
```

8.2 · Interfaz con Protocol (contrato estructural)

La segunda forma invierte la dirección de la dependencia, y es más sutil. Un **Protocol** es un contrato **estructural**: una clase lo cumple por el simple hecho de **tener los métodos que el protocolo pide, sin heredar de él ni enterarse de que existe**. Es 'duck typing verificable': si camina como pato y hace cuac, es pato, y `mypy` lo comprueba. Su superpoder práctico es enorme: permite tipar clases de **terceros**, que uno no puede modificar. Si una librería te da una clase con el método `a_json` pero que no hereda de nada tuyo, en el mundo de las interfaces nominales tendrías que escribir un adaptador; con un Protocol, esa clase ajena ya cumple el contrato, retroactivamente, sin tocarla.

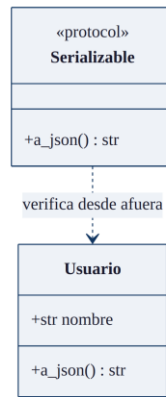


Figura 13 · Interfaz vía Protocol. La flecha punteada va del protocolo a la clase: verifica desde afuera; Usuario no lo conoce.

Python — interfaz vía Protocol

```

from typing import Protocol

class Serializable(Protocol):          # nadie hereda de esto
    def a_json(self) -> str: ...

class Usuario:                        # NO menciona Serializable
    def __init__(self, nombre: str) -> None:
        self.nombre = nombre
    def a_json(self) -> str:           # lo cumple por tener el método
        return '{"nombre": "' + self.nombre + '"}'

def exportar(x: Serializable) -> str: # mypy verifica que x tenga a_json()
    return x.a_json()

print(exportar(Usuario('Ana')))      # Usuario ya lo cumple, sin declararlo
  
```

ABC pura o Protocol — la regla que funciona

Elegí ABC pura (8.1) cuando **sos dueño** de las clases que cumplen el contrato y querés que la relación sea explícita y quede verificada al construir. El que cumple hereda y lo declara: hay un vínculo nominal, visible, deliberado.

Elegí Protocol (8.2) cuando el contrato **atraviesa código que no controlás** —librerías, clases ajenas— o cuando querés acoplar lo mínimo posible. La clase cumple el contrato sin saber que existe, y eso te da una flexibilidad que las interfaces nominales no pueden ofrecer.

Las dos formas conviven sin conflicto en un mismo sistema. La pregunta que las separa es siempre la misma que abrió esta sección: **¿el que cumple el contrato tiene que conocerlo, o alcanza con que tenga los métodos?** Si tiene que conocerlo, ABC. Si alcanza con que los tenga, Protocol.

Cierre · El mapa completo

Una tabla para tener al lado del teclado

Recorrido el catálogo, vale la pena mirarlo entero de una sola vez. La tabla que sigue reúne las diez entradas del catálogo —las ocho secciones, con las dos formas de dependencia y las dos de interfaz desglosadas— con tres datos por cada una: su símbolo en el diagrama UML, la marca por la que se la reconoce en el código Python, y la pregunta de diseño que decide cuándo usarla. Es la síntesis operativa

de todo lo anterior: la referencia rápida para el momento en que estás frente al modelo, dudando de qué relación tenés entre manos.

Relación	Símbolo	Marca en el código Python	Pregunta que la decide
Clase única	□	Sin refs a otras clases	¿Es autónoma?
Asociación	—	Ref mutua, creada afuera	¿Se conocen como iguales?
Agregación	◇—	Parte entra por parámetro	¿La parte sobrevive al todo?
Composición	◆—	Parte creada en el ctor	¿La parte muere con el todo?
Clase abstracta	«abstract»	ABC + @abstractmethod	¿Falta completar operaciones?
Herencia	—>	class Sub(Super) + super()	¿El dominio afirma 'es-un'?
Dependencia uso	«use» ->	Param de método, no guardado	¿Se usa y se suelta?
Dependencia creación	«create» ->	Parte(...) en método, devuelto	¿Se fabrica y se entrega?
Interfaz (ABC)	«interface»	ABC pura + herencia	¿El que cumple lo declara?
Interfaz (Protocol)	«protocol»	Protocol, sin herencia	¿Cumple sin enterarse?

La idea que unifica todo el manual

Si hubiera que llevarse una sola frase, sería esta. Asociación, agregación, composición y las dos dependencias **se ven casi iguales en el código** —una referencia guardada en un atributo, o un objeto que pasa fugazmente por un método—. Lo que las distingue no es la sintaxis, sino **el ciclo de vida**: quién crea la parte, quién la destruye, y cuánto tiempo la conserva el todo.

El diagrama UML captura esa decisión con un símbolo —un rombo, una flecha, una línea—. El código Python la captura con algo más humilde pero igual de preciso: **el lugar donde aparece el constructor de la parte**. ¿Se crea afuera y se recibe? ¿Se fabrica adentro y se guarda? ¿Se fabrica adentro y se suelta? Leé los constructores, y sabrás exactamente qué relación estás mirando. Ese es, al final, el oficio que este manual quiso enseñar.